SIMATS ENGINEERING

ASSESSMENT TOOL 1

COURSE NAME: OPERATING SYSTEMS COURSE CODE: CSA04

COURSE OUTCOME COVERED

CO2: Analyze process management and deadlock handling techniques to improve CPU utilization and system performance(BL3-BL4)

Assessment Tool Weightage: 40%

QUESTIONS: 10 Total Marks:50

Annexure A

Analytical Problem Solving

Code of Conduct:

I SIVA SUBRAMANIYAM B/192572089(Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

Operating System Process Scheduling & System Analysis

1. Airport Check-In Counter Scheduling

Example Scenario

Consider five passengers arriving at a check-in counter during peak hours:

- **P1:** Regular passenger (Arrival = 0 ms, Burst = 8 ms, Priority = 3)
- **P2:** Regular passenger (Arrival = 1 ms, Burst = 4 ms, Priority = 3)
- **P3:** Senior citizen needing assistance (Arrival = 2 ms, Burst = 9 ms, Priority = 1 — Highest)
- **P4:** Regular passenger (Arrival = 3 ms, Burst = 2 ms, Priority = 3)
- **P5:** Passenger with infant (Arrival = 4 ms, Burst = 5 ms, Priority = 2)

Execution & Analysis

First-Come, First-Served (FCFS)

Processes execute strictly in order of arrival.

Gantt Chart (FCFS):

| P1 (0-8) | P2 (8-12) | P3 (12-21) | P4 (21-23) | P5 (23-28) |

0 8 12 21 23 28

- **P1:** Completion = 8 | Turnaround = 8 - 0 = **8** | Waiting = 8 - 8 = **0**
- **P2:** Completion = 12 | Turnaround = 12 - 1 = **11** | Waiting = 11 - 4 = **7**
- **P3:** Completion = 21 | Turnaround = 21 - 2 = **19** | Waiting = 19 - 9 = **10**
- **P4:** Completion = 23 | Turnaround = 23 - 3 = **20** | Waiting = 20 - 2 = **18**
- **P5:** Completion = 28 | Turnaround = 28 - 4 = **24** | Waiting = 24 - 5 = **19**
- **Average Turnaround Time:** $(8 + 11 + 19 + 20 + 24)/5 = \mathbf{16.4}$ ms
- **Average Waiting Time:** $(0 + 7 + 10 + 18 + 19)/5 = \mathbf{10.8}$ ms

Non-Preemptive Priority Scheduling

When a counter becomes free, the waiting passenger with the highest priority (lowest numerical value) is served next.

Gantt Chart (Priority):

| P1 (0-8) | P3 (8-17) | P5 (17-22) | P2 (22-26) | P4 (26-28) |

0 8 17 22 26 28

- **P1:** Completion = 8 | Turnaround = **8** | Waiting = **0**
- **P3:** Completion = 17 | Turnaround = 17 - 2 = **15** | Waiting = 15 - 9 = **6**
- **P5:** Completion = 22 | Turnaround = 22 - 4 = **18** | Waiting = 18 - 5 = **13**
- **P2:** Completion = 26 | Turnaround = 26 - 1 = **25** | Waiting = 25 - 4 = **21**
- **P4:** Completion = 28 | Turnaround = 28 - 3 = **25** | Waiting = 25 - 2 = **23**
- **Average Turnaround Time:** $(8 + 15 + 18 + 25 + 25)/5 = \mathbf{18.2}$ ms
- **Average Waiting Time:** $(0 + 6 + 13 + 21 + 23)/5 = \mathbf{12.6}$ ms

Efficiency Analysis

- **FCFS** yields a lower aggregate average waiting time in this specific trace, but it treats all passengers uniformly, causing high-priority passengers (like P3) to suffer long delays (**10 ms waiting**).
- **Priority Scheduling** provides superior **service quality and equity relative to urgency**, reducing special assistance passenger wait time significantly (**from 10 ms down to 6 ms**). Thus, Priority Scheduling is far more effective for peak-hour airport operations.

2. Hospital Emergency Room (ER) Triage

Example Scenario

Four patients arrive continuously at the ER:

- **P1 (Non-Critical - Sprain):** Arrival = 0 min, Burst = 12 min, Priority = 3
- **P2 (Critical - Cardiac Arrest):** Arrival = 1 min, Burst = 8 min, Priority = 1 (Highest)
- **P3 (Non-Critical - Laceration):** Arrival = 2 min, Burst = 6 min, Priority = 3

- **P4 (Semi-Critical - Fracture):** Arrival = 3 min, Burst = 10 min, Priority = 2

Execution & Analysis

Preemptive Priority Scheduling

Critical patients preempt non-critical patients immediately upon arrival.

Gantt Chart (Preemptive Priority):

| P1 | P2 (1-9) | P4 (9-19) | P1 (19-30) | P3 (30-36) |

0 1 9 19 30 36

(P1 runs 1 min, gets preempted by P2 at t=1. P1 finishes remaining 11 min at t=19.)

- **P2 (Critical) Response Time:** $1 - 1 = \mathbf{0}$ min(Treated instantly upon arrival)
- **P4 (Semi-Critical) Response Time:** $9 - 3 = \mathbf{6}$ min
- **P1 (Non-Critical) Response Time:** $0 - 0 = \mathbf{0}$ min(Started initially before preemption)
- **P3 (Non-Critical) Response Time:** $30 - 2 = \mathbf{28}$ min

Round Robin (Time Quantum = 5 minutes)

Processes are treated in a simple cyclic queue without priority levels.

Gantt Chart (Round Robin Q=5):

| P1(0-5) | P2(5-10) | P3(10-15) | P4(15-20) | P1(20-27) | P2(27-30) | P3(30-31) | P4(31-36) |

0 5 10 15 20 27 30 31 36

- **P1 Response Time:** $0 - 0 = \mathbf{0}$ min
- **P2 (Critical) Response Time:** $5 - 1 = \mathbf{4}$ min(Waited 4 minutes behind lower priority P1!)
- **P3 Response Time:** $10 - 2 = \mathbf{8}$ min
- **P4 Response Time:** $15 - 3 = \mathbf{12}$ min

Response Time Comparison

Patient Type	Preemptive Priority Response Time	Round Robin (Q=5) Response Time
Critical (P2)	**0 min** (Immediate)	**4 min** (Dangerous Delay)
Non-Critical (P3)	**28 min**	**8 min**

Takeaway: Round Robin introduces unacceptable risks in emergency environments because it ignores urgency. **Preemptive Priority Scheduling** is strictly required to guarantee zero-delay response times for critical care.

3. Cloud Service Provider Task Scheduling

Example Scenario

Consider 10 computational cloud tasks arriving at time $t = 0$:

Task	Burst Time (ms)	Task	Burst Time (ms)
T1	15	**T6**	12
T2	3	**T7**	18
T3	8	**T8**	5
T4	2	**T9**	20
T5	10	**T10**	4

Analysis & Comparative Metrics

1. FCFS (First-Come, First-Served)

Tasks execute in exact sequential order T1 through T10.

- **Total Execution Time:** $15 + 3 + 8 + 2 + 10 + 12 + 18 + 5 + 20 + 4 = \mathbf{97}$ ms
- **Waiting Times:** T1:0, T2:15, T3:18, T4:26, T5:28, T6:38, T7:50, T8:68, T9:73, T10:93
- **Average Waiting Time:** $(0 + 15 + 18 + 26 + 28 + 38 + 50 + 68 + 73 + 93)/10 = \mathbf{42.3}$ ms

2. SJF (Shortest Job First - Non-Preemptive)

Sorted execution order by burst time: T4(2), T2(3), T10(4), T8(5), T3(8), T5(10), T6(12), T1(15), T7(18), T9(20).

- **Waiting Times:** T4:0, T2:2, T10:5, T8:9, T3:14, T5:22, T6:32, T1:44, T7:59, T9:77
- **Average Waiting Time:** $(0 + 2 + 5 + 9 + 14 + 22 + 32 + 44 + 59 + 77)/10 = \mathbf{26.4}$ ms

3. Round Robin (Time Quantum = 5 ms)

Tasks are context-switched every 5 ms.

- **Average Waiting Time:** $\approx \mathbf{48.6}$ ms(Higher due to frequent context-switch rotations).

CPU Utilization

In all three algorithms, since tasks run back-to-back without idle CPU gaps:

$$\text{CPU Utilization} = \frac{\text{Total Busy Time}}{\text{Total Time}} = \frac{97}{97} \times 100\% = \mathbf{100}\%$$

Recommendation for Cloud Environment

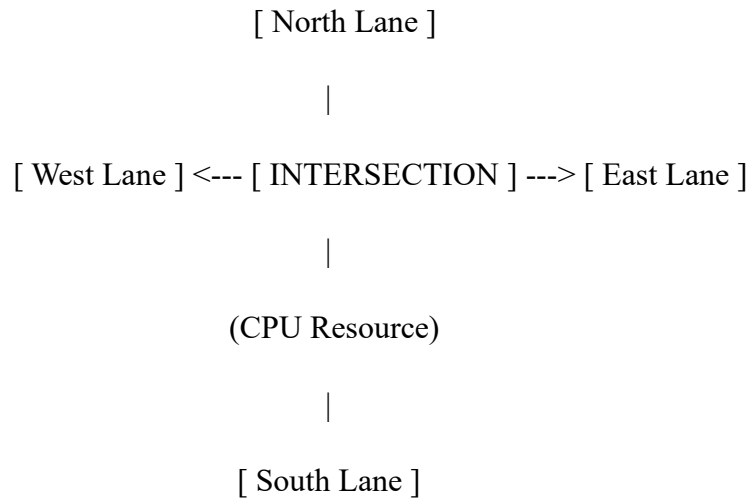
Algorithm	Average Waiting Time	CPU Utilization
FCFS	42.3 ms	100%
SJF	**26.4 ms** (Optimal)	**100%**
Round Robin (Q=5)	48.6 ms	100%

Recommendation: SJF (Shortest Job First) provides the absolute minimal average waiting time, maximizing throughput for cloud batch jobs. However, for interactive cloud APIs, a **Multilevel Feedback Queue (MLFQ)** is recommended in real production systems to eliminate long-job starvation while preserving low latency for short tasks.

4. Smart City Traffic Light Optimization

System Design Strategy

Traffic control at a smart intersection can be modeled by treating each traffic lane as a **Process** and the green light signal as the **CPU**.



To minimize average vehicle waiting time while guaranteeing safety:

1. **Dynamic Round Robin with Adaptive Time Quantum:** Base lane signal allocation uses Round Robin, but the time quantum is calculated dynamically based on real-time vehicle density (detected via computer vision sensors).
2. **Preemptive Priority Override:** Emergency vehicles (ambulances, fire engines, police) carry RFID/DSRC transponders that issue an absolute high-priority interrupt to preempt standard signal cycles.

Effectiveness Comparison

Feature	Round Robin (Standard)	Priority Scheduling (Preemptive)
Normal Traffic Flow	**Excellent:** Fair distribution; prevents any single lane from suffering infinite delays.	**Poor:** Low-density lanes starve if high-density lanes hold constant priority.
Emergency Vehicles	**Dangerous:** Emergency vehicles must wait for current lane quanta to expire.	**Optimal:** Instant preemptive signal change to green for the emergency lane.
Adaptability	Moderate (if static quantum).	High for critical events, poor for general flow.

Conclusion: The optimal strategy is a **Hybrid Multilevel Queue System**: Standard traffic operates under **Adaptive Round Robin**, while emergency vehicles trigger a **Preemptive Priority Interruption**, creating an instant green corridor.

5. Online Examination System Processing

Scenario & Data Table

Eight student exam submission requests arrive concurrently at $t = 0$:

Request ID	Arrival Time (ms)	Burst Time (ms)	Priority
R1	0	6	2
R2	0	2	1 (High)
R3	0	8	4
R4	0	3	3
R5	0	1	2
R6	0	4	1 (High)
R7	0	10	5
R8	0	5	3

FCFS Scheduling Results

Execution order: **R1 →R2 →R3 →R4 →R5 →R6 →R7 →R8**

Request	Burst Time	Completion Time	Turnaround Time (CT - AT)
R1	6	6	6
R2	2	8	8
R3	8	16	16
R4	3	19	19
R5	1	20	20
R6	4	24	24
R7	10	34	34
R8	5	39	39

Averages	—	—	**20.875 ms**

SJF (Shortest Job First) Scheduling Results

Execution order by burst time: **R5(1) →R2(2) →R4(3) →R6(4) →R8(5) →R1(6) →R3(8) →R7(10)**

Request	Burst Time	Completion Time	Turnaround Time (CT - AT)
R5	1	1	1
R2	2	3	3
R4	3	6	6
R6	4	10	10
R8	5	15	15
R1	6	21	21
R3	8	29	29
R7	10	39	39
Averages	—	—	**15.5 ms**

6. Robotic Assembly Line Failure Recovery

Process State Transitions During Execution & Failure

1. **New:** The supervisory controller generates a robotic task (e.g., "Weld Chassis Joint A").
2. **Ready:** The task is loaded into memory and queued, waiting for an available robotic arm.
3. **Running:** A robotic arm claims the task and begins physical execution.
4. **Waiting (or Hardware Failure Halt):**
    - Normal: The robot waits for a sensor signal or part positioning.
    - Failure State: If Robot #2 experiences a mechanical failure, a hardware interrupt is issued. The OS moves the running process out of the **Running** state.

5. **Rescheduling / Fault Recovery:** The OS updates the state of the incomplete task from **Running** back to **Ready**, re-assigning it to the queue of an operational robot (e.g., Robot #3).
6. **Terminated:** Once the task completes successfully, it enters the terminated state, releasing allocated resources.

Continuous Production Guarantee

By decoupling tasks from physical hardware via OS process queues:

- The system maintains heartbeat signals for every robot.
- Upon robot failure, preemptive rescheduling redistributes pending assembly tasks across remaining healthy nodes without halting the factory assembly line.

7. Video Streaming Service Processing

Example Scenario

User requests arrive for video segment buffering:

- **P1 (Standard User):** Arrival = 0 ms, Burst = 10 ms, Priority = 3
- **P2 (Premium User):** Arrival = 1 ms, Burst = 4 ms, Priority = 1 (High)
- **P3 (Standard User):** Arrival = 2 ms, Burst = 6 ms, Priority = 3
- **P4 (Premium User):** Arrival = 3 ms, Burst = 2 ms, Priority = 1 (High)

Non-Preemptive Priority Scheduling

Execution Order: P1(0-10) →P2(10-14) →P4(14-16) →P3(16-22)

Gantt Chart:

| P1 (0-10) | P2 (10-14) | P4 (14-16) | P3 (16-22) |

0 10 14 16 22

- **P1:** CT = 10, TAT = 10, WT = **0**
- **P2:** CT = 14, TAT = 13, WT = **9**
- **P4:** CT = 16, TAT = 13, WT = **11**

- **P3:** CT = 22, TAT = 20, WT = **14**

Evaluation Criteria

- **Fairness:** Pure priority scheduling is **inherently unfair** to standard users because premium user requests continuously jump ahead in the ready queue.
- **Starvation:** If a continuous stream of Premium requests arrives, Standard users (P1, P3) will suffer **infinite starvation**, experiencing buffer underflows and frozen video playback.
- **Throughput:** Overall throughput remains high because the processor is constantly active, but throughput is heavily skewed toward premium user sessions.

8. ATM Concurrent Transaction Processing

Example Scenario

Ten transactions (T_1 to T_{10}) with varying execution times are queued at $t = 0$:

Transaction	Burst Time	Transaction	Burst Time
T1	8 ms	**T6**	7 ms
T2	4 ms	**T7**	3 ms
T3	12 ms	**T8**	5 ms
T4	2 ms	**T9**	6 ms
T5	10 ms	**T10**	1 ms

Time Quantum = 4 ms

Round Robin Execution Trace

Round 1 (0 to 29 ms)

- **T1:** runs 4 ms (4 left) $[t = 0 - 4]$
- **T2:** runs 4 ms (**0 remaining →Completes at** $t = 8$) $[t = 4 - 8]$
- **T3:** runs 4 ms (8 left) $[t = 8 - 12]$
- **T4:** runs 2 ms (**0 remaining →Completes at** $t = 14$) $[t = 12 - 14]$

- **T5:** runs 4 ms (6 left) $[t = 14 - 18]$
- **T6:** runs 4 ms (3 left) $[t = 18 - 22]$
- **T7:** runs 3 ms (**0 remaining →Completes at** $t = 25$) $[t = 22 - 25]$
- **T8:** runs 4 ms (1 left) $[t = 25 - 29]$

Continuation (29 to 58 ms)

- **T9:** runs 4 ms (2 left) $[t = 29 - 33]$
- **T10:** runs 1 ms (**Completes at** $t = 34$) $[t = 33 - 34]$
- **T1:** runs 4 ms (**Completes at** $t = 38$) $[t = 34 - 38]$
- **T3:** runs 4 ms (4 left) $[t = 38 - 42]$
- **T5:** runs 4 ms (2 left) $[t = 42 - 46]$
- **T6:** runs 3 ms (**Completes at** $t = 49$) $[t = 46 - 49]$
- **T8:** runs 1 ms (**Completes at** $t = 50$) $[t = 49 - 50]$
- **T9:** runs 2 ms (**Completes at** $t = 52$) $[t = 50 - 52]$
- **T3:** runs 4 ms (**Completes at** $t = 56$) $[t = 52 - 56]$
- **T5:** runs 2 ms (**Completes at** $t = 58$) $[t = 56 - 58]$

Results Table

Transaction	Burst Time	Completion Time	Turnaround Time (CT)
T1	8	38	38
T2	4	8	8
T3	12	56	56
T4	2	14	14
T5	10	58	58
T6	7	49	49

T7	3	25	25
T8	5	50	50
T9	6	52	52
T10	1	34	34
Averages	—	—	**38.4 ms**

9. Autonomous Vehicle Process Control

Process State Identification Across Driving Intervals

Time Interval	Obstacle Detect	GPS Nav	Speed Control	Lane Detect
t = 0-10 ms	RUNNING	READY	WAITING	READY
t = 10-20 ms	WAITING	RUNNING	READY	READY
t = 20-30 ms	RUNNING	WAITING	PREEMPTED	WAITING

1. **Obstacle Detection:** Transitions rapidly between **Running** and **Ready/Waiting** (sensor processing). When a pedestrian appears, it transitions to **Running** at maximum priority.
2. **GPS Navigation:** Operates mainly in **Ready** or low-priority **Running** states; yields to active control systems.
3. **Speed Control / Braking:** Remains in **Waiting** state awaiting telemetry inputs, but shifts immediately to **Running** when safety thresholds are breached.
4. **Lane Detection:** Periodically moves from **Waiting** (frame acquisition) to **Running** (image analysis).

Justification for Preemptive Scheduling

In autonomous vehicles, scheduling failure leads to physical accidents:

- Non-preemptive scheduling would allow a low-priority task (e.g., updating the map UI display) to lock the CPU while a collision is imminent.

- **Preemptive Hard Real-Time Scheduling (like Rate Monotonic or Earliest Deadline First)** ensures that critical safety tasks (obstacle detection and emergency braking) instantly preempt non-critical tasks within microseconds.

10. Satellite Systems Priority & Starvation Resolution

Simulation & Starvation Problem

Consider four satellite sub-systems:

- **P1 (Telemetry - Low):** Priority 4, Burst = 20 ms
- **P2 (Power Mgmt - Med):** Priority 3, Burst = 15 ms
- **P3 (Nav - High):** Priority 2, Burst = 10 ms
- **P4 (Emergency Comm - Critical):** Priority 1, Burst = 25 ms

If emergency communication tasks (P4) or orbital adjustment commands arrive continuously, lower priority tasks like Telemetry (P1) suffer from **Starvation** (Indefinite Blocking). Consequently, the ground station receives no health metrics from the satellite, even though the satellite is operational.

Solution: Aging Technique

Aging gradually increases the priority of processes that sit waiting in the ready queue for long durations.

$$\text{Effective Priority} = \text{Base Priority} - \left(\frac{\text{Wait Time}}{\alpha}\right)$$

(Where lower numerical values represent higher priority).

Priority Dynamics with Aging:

Priority Value

(Lower = Higher Priority)

^

4| P1 (Telemetry) Base Priority

3| \

2| \ (Aging over time)

1|----\----------> Priority Level threshold reached -> P1 Executes!

0+----------> Time

- **Result:** P1's priority slowly rises from 4 to 3, then 2, and eventually 1. Once its priority is high enough, it receives CPU time to send critical health telemetry back to earth, completely preventing starvation.

11. Non-Preemptive Shortest Job First (SJF) Calculations

Process Table

All processes arrive at time $t = 0$:

Process	Burst Time (ms)
P1	10
P2	4
P3	6
P4	2

a. Calculations for Each Process

Sorted order of execution: **P4 (2 ms) →P2 (4 ms) →P3 (6 ms) →P1 (10 ms)**

Gantt Chart:

| P4 (0-2) | P2 (2-6) | P3 (6-12) | P1 (12-22) |

0 2 6 12 22

- **P4:** Completion Time = 2 ms
    - Turnaround Time (TAT) $= 2 - 0 = \mathbf{2}$ ms
    - Waiting Time (WT) $= 2 - 2 = \mathbf{0}$ ms
- **P2:** Completion Time = 6 ms
    - Turnaround Time (TAT) $= 6 - 0 = \mathbf{6}$ ms
    - Waiting Time (WT) $= 6 - 4 = \mathbf{2}$ ms

- **P3:** Completion Time = 12 ms
    - Turnaround Time (TAT) $= 12 - 0 = \mathbf{12}$ ms
    - Waiting Time (WT) $= 12 - 6 = \mathbf{6}$ ms
- **P1:** Completion Time = 22 ms
    - Turnaround Time (TAT) $= 22 - 0 = \mathbf{22}$ ms
    - Waiting Time (WT) $= 22 - 10 = \mathbf{12}$ ms

b. Average Waiting Time & Turnaround Time

$$\text{Average Turnaround Time} = \frac{2 + 6 + 12 + 22}{4} = \frac{42}{4} = \mathbf{10.5} \text{ ms}$$

$$\text{Average Waiting Time} = \frac{0 + 2 + 6 + 12}{4} = \frac{20}{4} = \mathbf{5.0} \text{ ms}$$

c. CPU Utilization Interpretation

- **Does SJF improve CPU utilization compared to FCFS?** In a simple single-processor model without idle delays between arrivals, **both SJF and FCFS achieve 100% CPU utilization** because the total burst time (22 ms) is executed continuously.
- **Key Advantage of SJF:** While CPU utilization remains unchanged, SJF **dramatically lowers the average waiting time** (from 13 msin FCFS down to 5 msin SJF) by eliminating the convoy effect.

12. Round Robin Scheduling – Time Quantum Impact

Process Data

Processes arrive at $t = 0$:

Process	**Burst Time (ms)**
P1	20
P2	5
P3	13
P4	8

Time Quantum $Q = 4$ ms

a. Gantt Chart Construction

Gantt Chart (Q = 4 ms):

| P1 | P2 | P3 | P4 | P1 | P2 | P3 | P4 | P1 | P3 | P1 | P3 | P1 |

0 4 8 12 16 20 21 25 29 33 37 41 42 46

Detailed Execution Sequence:

1. **P1:** runs 0–4 (16 left)
2. **P2:** runs 4–8 (1 left)
3. **P3:** runs 8–12 (9 left)
4. **P4:** runs 12–16 (4 left)
5. **P1:** runs 16–20 (12 left)
6. **P2:** runs 20–21 (**Finishes at t=21**)
7. **P3:** runs 21–25 (5 left)
8. **P4:** runs 25–29 (**Finishes at t=29**)
9. **P1:** runs 29–33 (8 left)
10. **P3:** runs 33–37 (1 left)
11. **P1:** runs 37–41 (4 left)
12. **P3:** runs 41–42 (**Finishes at t=42**)
13. **P1:** runs 42–46 (**Finishes at t=46**)

b. Waiting Time & Turnaround Time Calculation

- **P1:** Completion = 46 ms
    - Turnaround Time $= 46 - 0 = \mathbf{46}$ ms
    - Waiting Time $= 46 - 20 = \mathbf{26}$ ms
- **P2:** Completion = 21 ms

- Turnaround Time $= 21 - 0 = \mathbf{21}$ ms
- Waiting Time $= 21 - 5 = \mathbf{16}$ ms

- **P3:** Completion = 42 ms
  - Turnaround Time $= 42 - 0 = \mathbf{42}$ ms
  - Waiting Time $= 42 - 13 = \mathbf{29}$ ms
- **P4:** Completion = 29 ms
  - Turnaround Time $= 29 - 0 = \mathbf{29}$ ms
  - Waiting Time $= 29 - 8 = \mathbf{21}$ ms

$$\text{Average Turnaround Time} = \frac{46 + 21 + 42 + 29}{4} = \frac{138}{4} = \mathbf{34.5} \text{ ms}$$

$$\text{Average Waiting Time} = \frac{26 + 16 + 29 + 21}{4} = \frac{92}{4} = \mathbf{23.0} \text{ ms}$$

c. Analysis of Changing Quantum to 10 ms

When $Q = 10$ ms:

- **Context Switches:** The total number of context switches drops significantly (from 12 switches down to 5 switches).
- **CPU Utilization: Improves.** Less CPU overhead is wasted saving and restoring registers during context switches.
- **Response Time: Degrades for short processes.** Short processes (like P2) wait longer to get their first CPU allocation.
- **Limiting Behavior:** As $Q \to \infty$, Round Robin degenerates into standard **FCFS**.

13. CPU Utilization – Multiprogramming Level Analysis

System Parameters

- I/O Wait Probability (p) = $80\% = 0.8$
- CPU Work Percentage = $20\% = 0.2$
- CPU Utilization Formula:

$$U = 1 - p^n$$

a. Calculation for Degree of Multiprogramming ($n = 4$)

$$U = 1 - (0.8)^4$$

$$(0.8)^4 = 0.4096$$

$$U = 1 - 0.4096 = 0.5904 \Longrightarrow \mathbf{59.04}\%$$

b. Calculation for Degree of Multiprogramming ($n = 6$)

$$U = 1 - (0.8)^6$$

$$(0.8)^6 = 0.262144$$

$$U = 1 - 0.262144 = 0.737856 \Longrightarrow \mathbf{73.79}\%$$

c. Interpretation & System Performance

- **Which configuration improves performance?** The configuration with $n =$ 6**processes** significantly improves system performance, raising CPU utilization from **59.04% to 73.79%** (a **14.75% absolute gain**).
- **Why?** Because each process spends 80% of its time waiting for I/O operations to complete, a system with a low degree of multiprogramming leaves the CPU idle most of the time. Increasing the degree of multiprogramming ensures that while some processes are blocked waiting for I/O, there is a much higher probability that at least one process is ready to use the CPU, keeping the processor productive.